```
# If none, then the current directory is used.
def main(argv):
    try:
        path=argv[1]
    except IndexError:
        path='.'
    es = Elasticsearch(hosts='h-mstr')
    id = 0
# index each document with 3 attributes:
# path, title (the file name) and text (the text content)
    for p,f in get_documents(path):
        text = process_file(p,f)
        doc = {'path':p, 'title':f, 'text':text}
        id += 1
        es.index(index='documents', doc_type='text',id=id,
body= doc)
if __name__=="__main__":
    main(sys.argv)
```

Once the index is created, you cannot increase the number of shards. However, you can change the replication value as follows:

```
$ curl  -XPUT  'h-mstr:9200/documents/_settings'  -d '
{
     "index" : {
         "number_of_replicas" : 1
     } }
     '
```

Now, the number of shards will be 7, 7 and 6, respectively, on the three nodes. As you would expect, if one of the nodes is down, you will still be able to search the documents. If more than one node is down, the search will return a partial result from the shards that are still available.

Searching the data

The program *search_documents.py* below uses the *query_string* option of Elasticsearch to search for the string passed as a parameter in the content field 'text'. It returns the fields 'path' and 'title' in the response, which are combined to print the full file names of the documents found.

```
#!/usr/bin/python
import sys
from elasticsearch import Elasticsearch
def main(query_string):
    es = Elasticsearch(['h-mstr'])
    query_body = {'query':
              {'query_string':
               { 'default_field':'text',
                 'query': query_string}},
              'fields':['path','title']
              }
    # response is a dictionary with nested dictionaries
    response = es.search(index='documents', body=query_body)
    for hit in response['hits']['hits']:
        print '/'.join(hit['fields']['path'] + hit['fields']
['title'])
# run the program with search expression as a parameter
if __name__=='__main__':
    main(' '.join(sys.argv[1:]))
```

You can now search using expressions like the following:

```
$ python search_documents.py smalltalk objects
$ python search_documents.py smalltalk  AND objects
$ python search_documents.py +smalltalk  objects
$ python search_documents.py +smalltalk  +objects
```

More details can be found at the Lucene and Elasticsearch sites.

Open source options let you build a custom, scalable search engine. You may include information from your databases, documents, emails, etc, very conveniently. Hence, it is a shame to come across sites that do not offer an easy way to search their content. One hopes that website managers will add that functionality using tools like Elasticsearch! END

By: Anil Seth

The author has earned the right to do what interests him. You can find him online at *http://sethanil.com*, *http://sethanil.blogspot.com*, and reach him via email at *anil@sethanil.com*